

WebLens AI

Analyze. Understand. Improve.

A deterministic website quality engine, explained by generative AI.

32

rule checks

5,308

lines of code

3

AI providers

58

source files

Two engines. One report.

WebLens is not a chatbot pointed at a website. Measurement and interpretation are deliberately separated.

1 Rule engine

Deterministic

- Fetches and parses the served HTML
- Applies 32 fixed checks
- Assigns severity and a point penalty
- Computes every score

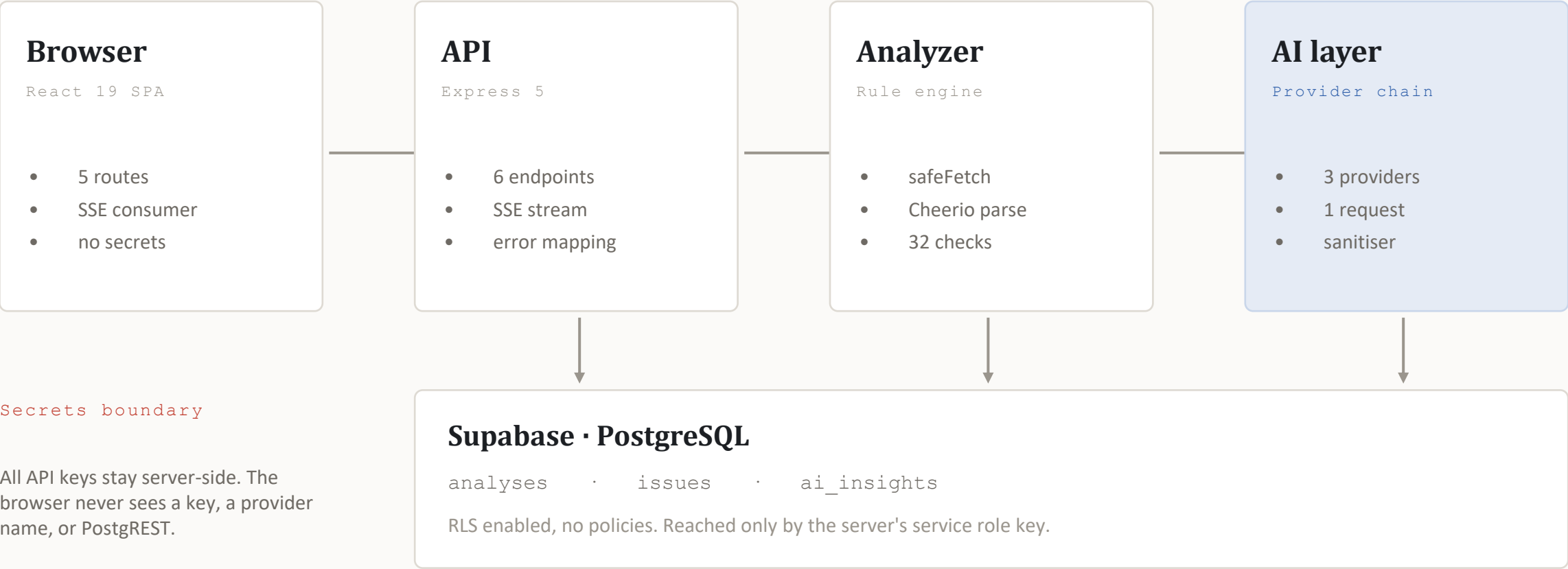
2 AI engine

Interpretive

- Receives findings, never raw HTML
- Explains real-world impact
- Suggests before / after code fixes
- Never touches a score

Consequence: identical HTML always produces an identical score, and the report still works when every AI provider is down.

Architecture



Stack

1 Frontend

React

19.3

Vite

7.2

Tailwind CSS

4.1

React Router

8.3

2 Motion

GSAP

3.15

Motion

13.2

Lenis

1.3

Lucide icons

1.44

3 Backend

Node.js

20+

Express

5.1

Cheerio

1.1

dotenv

17.2

4 Data & AI

Supabase JS

2.58

Groq

gpt-oss-120b

Gemini

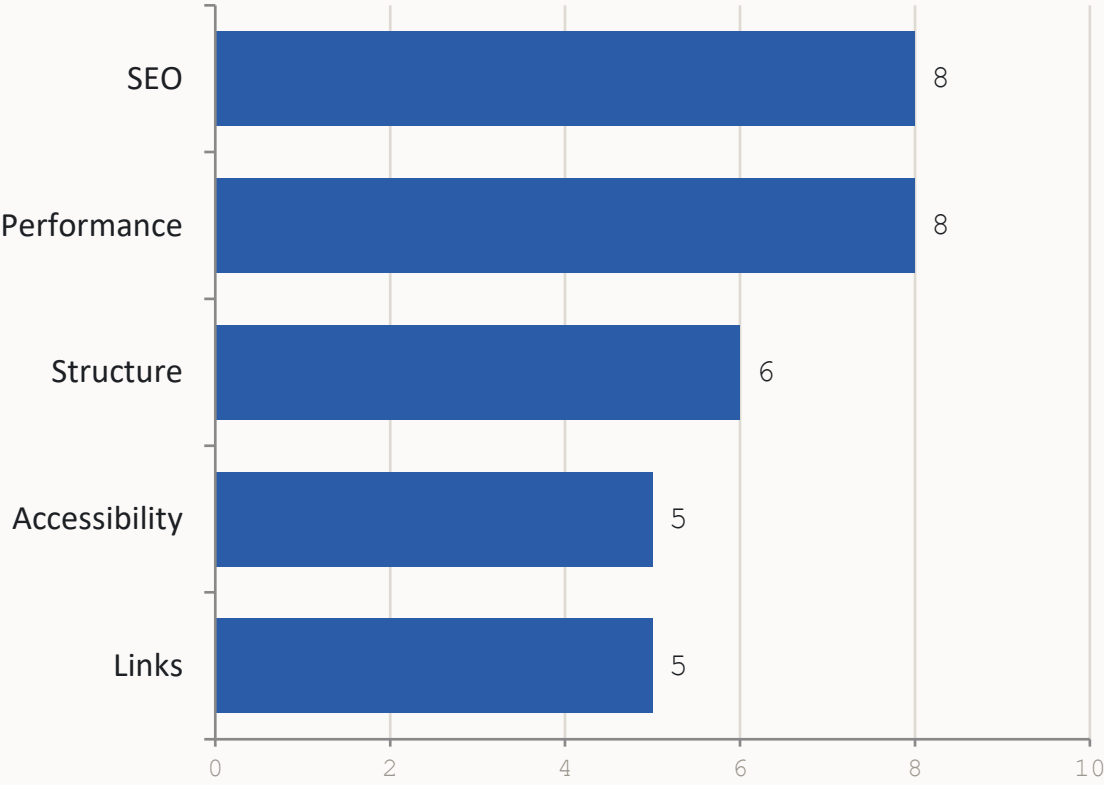
2.5-flash

OpenAI

gpt-4o-mini

No UI component library and no state library. 5,308 lines across 58 source files.

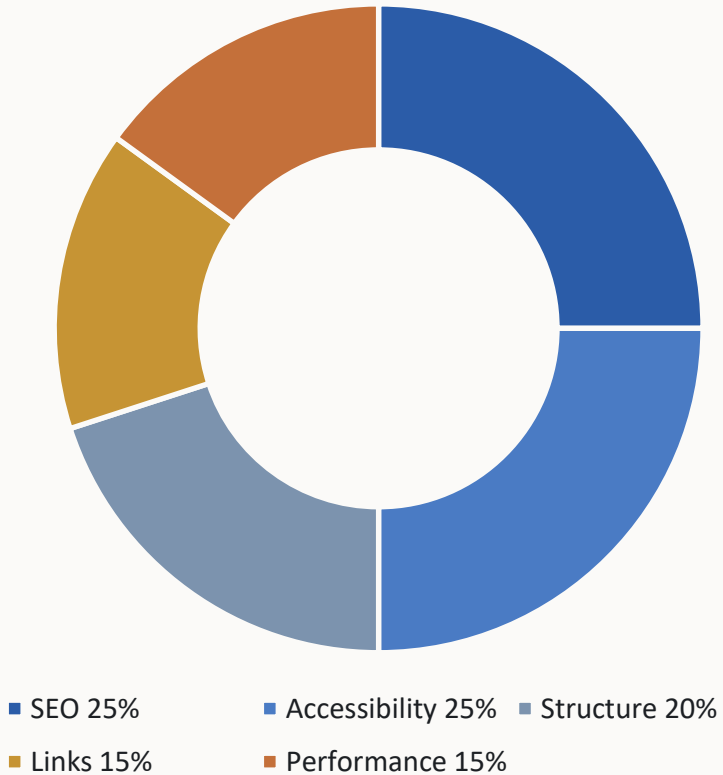
32 checks across five categories



- **SEO**
Missing title, description, viewport, canonical, lang, Open Graph
- **Performance**
Render-blocking scripts, unsized images, oversized DOM
- **Structure**
Absent or duplicated H1, skipped heading levels, missing landmarks
- **Accessibility**
Images without alt text, unlabelled form fields, buttons with no accessible name
- **Links**
Empty or malformed hrefs, target=_blank without rel=noopener

Every finding carries a severity, a location, evidence from the page, and a fixed point penalty.

Explainable by construction



$$\text{overall} = \sum (\text{category} \times \text{weight})$$

Each category starts at 100 and loses each finding's penalty.

1

Deterministic

The same HTML always yields the same number. No randomness, no model input.

2

Traceable

Every point deducted maps to one named finding the user can read.

3

Independent

Computed before the AI layer is called, and unaffected by whether it succeeds.

One analysis, end to end

01

Fetching website

URL validated, DNS resolved, private ranges rejected

02

Analyzing HTML

Cheerio parse. No render, no script execution

03

Checking SEO

8 checks

04

Checking accessibility

5 checks

05

Analyzing links

5 checks

06

Evaluating structure

6 checks + 8 performance signals

07

Calculating score

Weighted mean, persisted to PostgreSQL

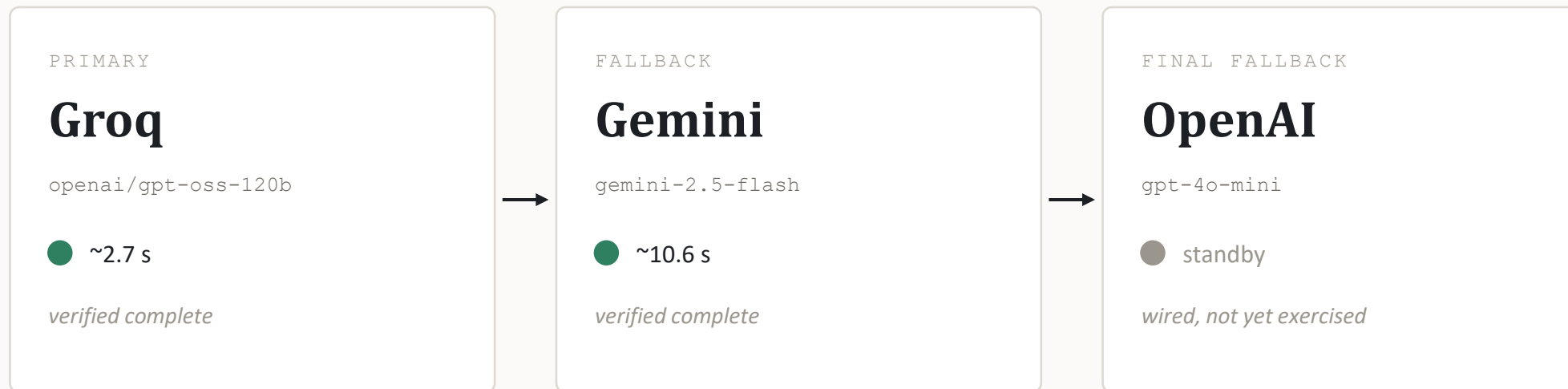
08

Generating AI insights

One request, cached by findings hash

Progress is streamed over Server-Sent Events. Each stage event is emitted as that stage actually begins — there is no timer and no interpolated percentage.

Three-provider failover



Switch conditions

429 · 4xx · 5xx · timeout · network error

Advances immediately. The user is never asked to retry.

If all three are exhausted

The deterministic report is returned in full, labelled as being without AI insight. Analysis has never depended on a model being reachable.

The model is not trusted by default

Prompt instructions drift. Every generation is validated in code before it can reach the report.

1

Invented findings are dropped

Every issue id in priorities, explanations and code fixes is checked against the rule engine's output.

2

Roadmaps are rebuilt

One step per real finding. Padding such as “run an SEO crawl” maps to no finding and is discarded.

3

Priority is derived, not accepted

A low-severity finding renders as low priority regardless of how urgent the model wanted to sound.

4

Ordering is enforced

Steps are sorted by the engine's severity ranking and renumbered.

A fabricated finding must never appear beside a measured one.

Data model and cost control

analyses

id, url, http_status
overall_score
5 category scores
statistics (jsonb)
created_at

issues

analysis_id (fk)
rule_id, category
severity, weight
title, description
evidence, recommendation

ai_insights

analysis_id (fk)
findings_hash
provider, model
summary, priorities
code_fixes, roadmap

RLS

Enabled on all three tables with no policies. A leaked anon key grants nothing.

AI response caching

```
findings_hash = SHA-256( url + category scores + sorted issue ids + INSIGHT_VERSION )
```

Keyed on findings rather than URL: re-analysing an unchanged page costs nothing, while a genuinely changed page regenerates. One AI request per analysis, never one per issue.

Fetching arbitrary user-supplied URLs

Accepting any URL from the public internet is the application's largest attack surface.

1

SSRF guard

Hosts are DNS-resolved and checked before connecting. Private, loopback, link-local and CGNAT ranges are rejected.

2

Redirects revalidated

Every hop is re-checked, so a public host cannot redirect the fetcher to 127.0.0.1.

3

Resource ceilings

3 MB and 12 seconds, streamed against a byte counter rather than a declared content-length.

4

Secret isolation

Keys live only in server/.env. The client bundle contains no key and no provider name.

5

No raw errors

Failures map to plain messages. Stack traces never reach the user.

Motion that cannot strand content

The problem

GSAP and Motion animate from opacity 0. When frame delivery stalls — a background tab, a loaded machine — content is left invisible, or a score counter reads 8 while tweening toward 82. A wrong number shown confidently is worse than no animation.

The solution

Every animated block declares its finished state as a function. `runAnimation` applies it immediately under reduced motion or a hidden document, and enforces a wall-clock deadline otherwise. List entrances use CSS animation, whose timing is wall-clock and cannot be stranded.

Lenis watchdog

Smooth scroll is removed automatically if its frame loop does not initialise within one second, restoring native scrolling.

Accessibility

Semantic landmarks, skip link, visible focus rings, live-region progress, and full `prefers-reduced-motion` support.

Performance

Report and chart components are lazily loaded. Production bundle is 193 KB gzipped.

What I built

01

Rule engine

32 deterministic checks, severity model and weighted scoring, written as pure functions with a fixture-based test suite.

03

AI integration

Three-provider failover, single-request prompting, findings-hash caching, and a sanitiser that constrains model output.

05

Design system

Token-driven visual language: one accent, semantic severity colours, editorial typography, built without a component library.

02

API & streaming

Six Express endpoints, SSE progress reflecting genuine analyzer stages, and friendly mapping for every failure mode.

04

Database

PostgreSQL schema on Supabase, RLS posture, and a repository layer that degrades without taking analysis down.

06

Frontend

Five routes, the report experience, and motion architecture with correctness guarantees under degraded rendering.

Verification: two offline test suites (rule engine, failover chain) plus a live provider check that grades every response for completeness.

Precision over spectacle.

Measure first. Explain clearly. Improve intelligently.

Honest

Never reports a measurement it did not take

Reproducible

The engine owns the score, not the model

Resilient

Fully useful with every AI provider offline

`github.com/preetmendpara/WebLensAi`